Polytech Sorbonne

Spécialité électronique informatique
4ème année en alternance

Rapport de TP 2

Programmes client/serveur pour l'identification

Module : Système d'Exploitation et Réseau

Auteurs :
Jules Gueneau

Date :
8 avril 2026

Table des matières

1 Mise en place du protocole BEUIP v1 **2**
1.1 2.1 et 2.2 : Création du serveur et gestion du broadcast 2
1.2 2.3 : Création du client pour les tests 2
1.3 2.3.1 : Commande d'affichage de la liste 2
1.4 2.3.2 : Commande de message privé (unicast) 3
1.5 2.3.3 : Commande de message à tout le monde (broadcast applicatif) 3
1.6 3.1 : Restructuration du code et librairie `creme` 3
1.7 3.2 et 3.3 : Gestion du cycle de vie du serveur (`start` et `stop`) 4
1.8 3.4 : Implémentation de la commande unifiée `mess` 5

1 Mise en place du protocole BEUIP v1

1.1 2.1 et 2.2 : Création du serveur et gestion du broadcast

Le développement a débuté par la mise en place d'un serveur UDP (`servbeuip`) capable de s'annoncer sur le réseau via une adresse de diffusion (broadcast). Le serveur écoute sur le port 9998 et enregistre les identifiants des pairs dans une table locale. La trace ci-dessous montre la compilation avec le drapeau de débogage (`-DTRACE`) et le démarrage du serveur qui s'auto-détecte sur l'interface réseau.

```
UDP — servbeuip JUL — 80×24

julesgueneau@MacBook-Air-de-Jules ~ % cd Desktop
julesgueneau@MacBook-Air-de-Jules Desktop % cd Polytech_Sorbonne/Semestre_4/OS
julesgueneau@MacBook-Air-de-Jules OS % ls
Jules_GUENEAU_TP1.pdf   QCM_1.pdf               QCM_3.pdf
Polytech_OsUserTP1.pdf  QCM_2.pdf               TP1_OS
julesgueneau@MacBook-Air-de-Jules OS % cd TP
julesgueneau@MacBook-Air-de-Jules TP % ls
UDP
julesgueneau@MacBook-Air-de-Jules TP % cd u
cd: no such file or directory: u
julesgueneau@MacBook-Air-de-Jules TP % cd UDP
julesgueneau@MacBook-Air-de-Jules UDP % ls
cliudp          cliudp~make     servudp
cliudp.c        Makefile        servudp.c
julesgueneau@MacBook-Air-de-Jules UDP %             strcpy(sender_pseudo, buf +
6);

zsh: missing end of string
julesgueneau@MacBook-Air-de-Jules UDP % cc -Wall -DTRACE -o servbeuip servbeuip.
c
julesgueneau@MacBook-Air-de-Jules UDP % ./servbeuip JUL
serveur demarre sur le port 9998 avec le pseudo : JUL
nouveau contact : JUL (192.168.88.189)
```

FIGURE 1 – Démarrage du serveur et détection du premier contact.

1.2 2.3 : Création du client pour les tests

Afin d'interagir avec le serveur, un client (`clibeuip`) a été développé. Il envoie des datagrammes à l'adresse de boucle locale (`127.0.0.1`) pour des raisons de sécurité imposées par l'architecture. La capture suivante illustre l'envoi d'une commande d'annonce (code '1') par le client, interceptée et traitée par le serveur local.

```
julesgueneau@MacBook-Air-de-Jules UDP % ./servbeuip JUL
serveur demarre sur le port 9998 avec le pseudo : JUL
nouveau contact : JUL (192.168.88.189)
nouveau contact : TEST_LOCAL (127.0.0.1)
```

```
utilisation : ./clibeuip code pseudo
codes : 1=broadcast, 2=ack
julesgueneau@MacBook-Air-de-Jules UDP % ./clibeuip 1 TEST_LOCAL
commande 1 envoyee au serveur local
```

FIGURE 2 – Envoi d'une annonce via le client de test.

1.3 2.3.1 : Commande d'affichage de la liste

La commande interne associée au code '3' permet d'afficher l'état de la table de routage interne du serveur. À la réception de cette requête locale, le serveur parcourt son tableau de structures et affiche les adresses IP et les pseudos qui y sont associés.

```
julesgueneau@MacBook-Air-de-Jules UDP % ./servbeuip JUL
serveur demarre sur le port 9998 avec le pseudo : JUL
nouveau contact : JUL (192.168.88.189)

--- liste des presents (1) ---
ip : 192.168.88.189 | pseudo : JUL
------------------------------
```

FIGURE 3 – Affichage de la table des contacts suite au code '3'.

1.4 2.3.2 : Commande de message privé (unicast)

L'implémentation du routage de messages privés utilise le code '4'. Le client transmet le pseudo du destinataire et le message, séparés manuellement par un caractère nul (`\0`) dans le buffer. Le serveur relaie ensuite ce contenu en utilisant le code '9' vers l'adresse IP correspondante trouvée dans sa table.

```
^C
julesgueneau@MacBook-Air-de-Jules UDP % ./servbeuip JUL
serveur demarre sur le port 9998 avec le pseudo : JUL
nouveau contact : JUL (192.168.88.189)
nouveau contact : TEST_LOCAL (127.0.0.1)
Message de JUL : Salut
Message de TEST_LOCAL : Salut
```

FIGURE 4 – Test de réception d'un message privé relayé par le serveur.

1.5 2.3.3 : Commande de message à tout le monde (broadcast applicatif)

Le code '5' implémente une diffusion applicative. À sa réception, le serveur itère sur l'ensemble de ses contacts enregistrés (à l'exclusion de lui-même) et génère un datagramme individuel de type '9' pour chaque pair. La capture montre le serveur détectant plusieurs pairs sur le réseau physique du TP et le client envoyant la commande de diffusion globale.

```
serveur JUL en ligne
nouveau contact : JUL (192.168.88.189)
nouveau contact : alice (192.168.88.137)
nouveau contact : Geoffrey (192.168.88.230)
nouveau contact : dede (192.168.88.137)
nouveau contact : alice (192.168.88.232)
message de alice : coucouuuu
nouveau contact : Remi (192.168.88.158)
nouveau contact : Ajinthini (192.168.88.238)
```

```
julesgueneau@MacBook-Air-de-Jules UDP % ./clibeuip 5 "Salut"
commande 5 relayee au serveur local
julesgueneau@MacBook-Air-de-Jules UDP % ./clibeuip 5 "Salut"
commande 5 relayee au serveur local
julesgueneau@MacBook-Air-de-Jules UDP % ./clibeuip 5 "Salut"
commande 5 relayee au serveur local
julesgueneau@MacBook-Air-de-Jules UDP % ./clibeuip 5 "Salut"
commande 5 relayee au serveur local
julesgueneau@MacBook-Air-de-Jules UDP % ./clibeuip 5 "Salut"
commande 5 relayee au serveur local
```

FIGURE 5 – Détection des pairs sur le réseau et envoi d'un message global.

1.6 3.1 : Restructuration du code et librairie `creme`

Afin de modulariser le projet et de préparer l'intégration dans l'interpréteur `biceps`, la logique réseau a été extraite pour former une librairie indépendante nommée `creme`. Le code de communication UDP est compilé en un fichier objet (`creme.o`). La vérification de la table des symboles à l'aide de l'utilitaire `nm -g` confirme l'exportation correcte des fonctions strictement nécessaires (indiquées par le marqueur 'T'), garantissant l'encapsulation de la couche de transport.

```
UDP — -zsh — 80×24
arret du serveur
julesgueneau@MacBook-Air-de-Jules UDP % make clean
rm -f *.o cliudp servudp servbeuip clibeuip
julesgueneau@MacBook-Air-de-Jules UDP % make
cc -Wall -o servudp servudp.c
cc -Wall -o cliudp cliudp.c
cc -Wall -c creme.c
cc -Wall -DTRACE -o servbeuip servbeuip.c creme.o
cc -Wall -o clibeuip clibeuip.c creme.o
julesgueneau@MacBook-Air-de-Jules UDP % nm -g creme.o
                 U ___sprintf_chk
                 U ___stack_chk_fail
                 U ___stack_chk_guard
                 U ___strcpy_chk
                 U _bind
0000000000000144 T _creme_addr_to_str
0000000000000000 T _creme_init_socket
00000000000001c0 T _creme_send_msg
0000000000000108 T _creme_set_broadcast
                 U _sendto
                 U _setsockopt
                 U _socket
                 U _strlen
```

```
UDP — -zsh — 80×24
SYMBOL TABLE:
0000000000000000 l     F __TEXT,__text ltmp0
00000000000000e4 l     F __TEXT,__text __OSSwapInt16
00000000000003e8 l     O __DATA,__bss _creme_addr_to_str.b
0000000000000328 l     O __TEXT,__cstring l_.str
0000000000000334 l     O __TEXT,__cstring l_.str.1
000000000000033d l     O __TEXT,__cstring l_.str.2
00000000000003e8 l     O __DATA,__bss ltmp1
0000000000000328 l     O __TEXT,__cstring ltmp2
0000000000000348 l     O __LD,__compact_unwind ltmp3
0000000000000144 g     F __TEXT,__text _creme_addr_to_str
0000000000000000 g     F __TEXT,__text _creme_init_socket
00000000000001c0 g     F __TEXT,__text _creme_send_msg
0000000000000108 g     F __TEXT,__text _creme_set_broadcast
0000000000000000         *UND* ___sprintf_chk
0000000000000000         *UND* ___stack_chk_fail
0000000000000000         *UND* ___stack_chk_guard
0000000000000000         *UND* ___strcpy_chk
0000000000000000         *UND* _bind
0000000000000000         *UND* _sendto
0000000000000000         *UND* _setsockopt
0000000000000000         *UND* _socket
0000000000000000         *UND* _strlen
```

FIGURE 6 – Compilation modulaire et vérification des symboles exportés avec `nm`.

1.7 3.2 et 3.3 : Gestion du cycle de vie du serveur (`start` et `stop`)

Le shell `biceps` a été enrichi de la commande interne `beuip` pour piloter directement le processus d'écoute. La commande `beuip start pseudo` duplique le processus courant via l'appel système `fork`, puis utilise `execl` pour charger l'exécutable du serveur en tâche de fond. Inversement, la commande `beuip stop` utilise la primitive `kill` pour émettre un signal d'interruption au processus fils. Le gestionnaire de signaux du serveur intercepte cette requête, diffuse un message de déconnexion (code '0') sur le réseau, puis libère proprement les ressources allouées.

```
julesgueneau@MacBookAir.TPOSUSER$ lsof -p 63306
COMMAND   PID         USER   FD   TYPE             DEVICE SIZE/OFF
  NODE NAME
biceps  63306 julesgueneau  cwd    DIR               1,16      768            81
904199 /Users/julesgueneau/Desktop/Polytech_Sorbonne/Semestre_4/OS/TP/UDP
biceps  63306 julesgueneau  txt    REG               1,16    52920            81
921368 /Users/julesgueneau/Desktop/Polytech_Sorbonne/Semestre_4/OS/TP/UDP/biceps
biceps  63306 julesgueneau  txt    REG               1,16  2300384 1152921500312
569191 /usr/lib/dyld
biceps  63306 julesgueneau    0u   CHR               16,0  0t20695
  1909 /dev/ttys000
biceps  63306 julesgueneau    1u   CHR               16,0  0t20695
  1909 /dev/ttys000
biceps  63306 julesgueneau    2u   CHR               16,0  0t20695
  1909 /dev/ttys000
biceps  63306 julesgueneau    4   PIPE 0x71efa8c502700356    16384

biceps  63306 julesgueneau    5   PIPE 0x28206aa0f5e9892c    16384

biceps  63306 julesgueneau    6   PIPE 0x7f3e63f6a8e03e88    16384

julesgueneau@MacBookAir.TPOSUSER$ kill 63306
zsh: terminated  ./biceps beuip start JUL
julesgueneau@MacBook-Air-de-Jules UDP %
```

FIGURE 7 – Analyse des descripteurs de fichiers du processus et terminaison par signal.

```
julesgueneau@MacBookAir.TPOSUSER$ beuip start Jules
julesgueneau@MacBookAir.TPOSUSER$ serveur Jules pret sur port 9998
nouveau contact : Jules (192.168.88.189)
nouveau contact : Max (192.168.88.188)
--- table des presents (2) ---
Jules : 192.168.88.189
Max : 192.168.88.188
--- table des presents (2) ---
Jules : 192.168.88.189
Max : 192.168.88.188
```

```
UDP — biceps — 80×24
julesgueneau@MacBookAir.TPOSUSER$ beuip list
commande 3 relayee au serveur local
julesgueneau@MacBookAir.TPOSUSER$
```

FIGURE 8 – Exécution du serveur en arrière-plan et communication locale concurrente.

1.8 3.4 : Implémentation de la commande unifiée `mess`

Afin de s'affranchir du programme utilitaire `clibeuip`, une commande interne `mess` a été intégrée au cœur de l'interpréteur `biceps`. Elle centralise l'interface utilisateur pour la messagerie en gérant les requêtes d'état (`mess list`), l'envoi de messages privés (`mess pseudo message`) et les diffusions applicatives (`mess all message`). Les datagrammes sont construits via les appels à la librairie `creme` et expédiés sur l'interface locale de bouclage vers le port d'écoute du serveur.

```
   609c225..a9c4993  main -> main
[julesgueneau@MacBook-Air-de-Jules UDP % ./biceps
[julesgueneau@MacBookAir.TPOSUSER$ mess list
 commande 3 relayee au serveur local
 trace : commande mess executee avec succes
[julesgueneau@MacBookAir.TPOSUSER$ mess all salut
 commande 5 relayee au serveur local
 trace : commande mess executee avec succes
[julesgueneau@MacBookAir.TPOSUSER$ mess Jules test
 commande 4 relayee au serveur local
 trace : commande mess executee avec succes
[julesgueneau@MacBookAir.TPOSUSER$ mess all salut
 commande 5 relayee au serveur local
 trace : commande mess executee avec succes
[julesgueneau@MacBookAir.TPOSUSER$ beuip stop
```

FIGURE 9 – Pilotage de la messagerie directement depuis l'interpréteur `biceps`.

Du côté du serveur, les traces d'exécution, activées par la directive de compilation `-DTRACE`, confirment la bonne réception locale des directives et le routage physique approprié des datagrammes UDP finaux (transmission ou affichage console), validant ainsi l'architecture client/serveur globale. L'arrêt propre via l'interception du signal (signal 30 dans macOS/BSD pour SIGUSR1 ou approchant, ou SIGINT standard) est également démontré.

```
[julesgueneau@MacBook-Air-de-Jules UDP % ./biceps
[julesgueneau@MacBookAir.TPOSUSER$ beuip start Jules
 julesgueneau@MacBookAir.TPOSUSER$ serveur Jules pret sur port 9998
 nouveau contact : Jules (192.168.88.189)
 nouveau contact :  (192.168.88.236)
 --- table des presents (2) ---
 Jules : 192.168.88.189
  : 192.168.88.236
 depart de :
 message de Jules : test
 nouveau contact :  (192.168.88.236)
 message de  :

[julesgueneau@MacBookAir.TPOSUSER$ beuip stop

 arret du serveur beuip via signal 30
 serveur arrete
```

FIGURE 10 – Réception, traitement des messages et arrêt propre du processus serveur.